



UNIDAD III

PUNTEROS

Tipos de datos

Tipo	Ejemplo	Bytes	Rango
char	'a'	1	0..255
short	-15	1	-128..127
int	1024	2	-32768..32767
long	262144	4	-2147483648..2147483637
float	10.5	4	3.4×10^{-38} .. 3.4×10^{308}
double	0.00045	8	1.7×10^{-308} .. 1.7×10^{308}

Punteros

- ▶ Un puntero es una variable que puede *almacenar una dirección de memoria*.
- ▶ Una variable de tipo puntero puede guardar direcciones de variables de un tipo determinado: punteros a int, a double, a char, etc.

Declaración de punteros

tipo *nombre_variable;

int i=3, *p,*r;// p y r son punteros a entero
double d=3.3,*q; // q es un puntero a double
char c='a', *t;// t es un puntero a carácter

▶ p=&i;

▶ r=p;

▶ p=q; es un **ERROR** porque son punteros de diferente tipo

Operaciones con punteros

- ▶ `cin >> *p;`
- ▶ `if (p == r)`
- ▶ `cout << "El valor de i es: " << *p;`
- ▶ `*p = *p + 3;`
- ▶ `p = p + 3;`

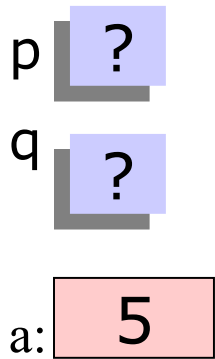
Mostrar una dirección

- ▶ Es posible desplegar una dirección como valor de una variable de tipo puntero, o la dirección de cualquier variable, utilizando `cout`

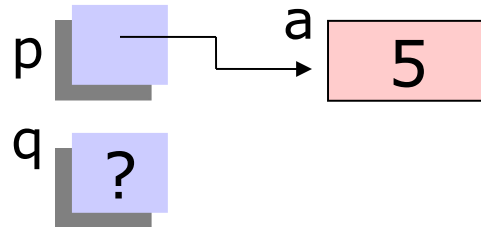
```
cout << "p = " << p << endl;  
cout << "&n = " << &n << endl;  
cout << "&p= " << &p << endl;
```

Ejemplo

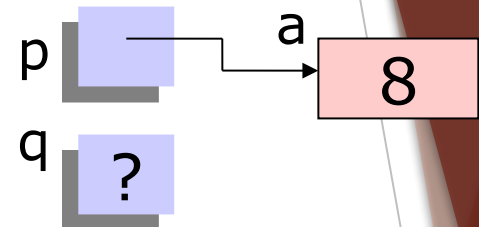
1. `int *p, *q, a=5`



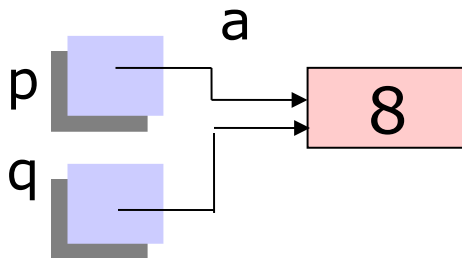
2. `p=&a`



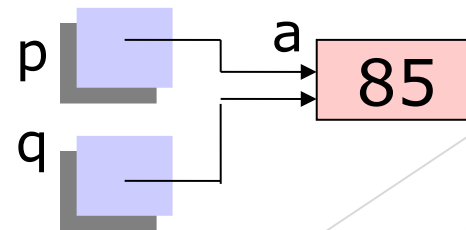
3. `*p=8;`



4. `q=p;`

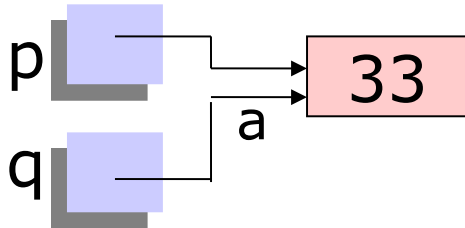


5. `*q=85;`

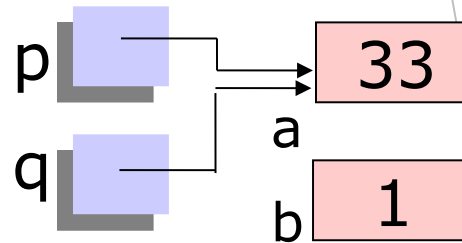


Ejemplo

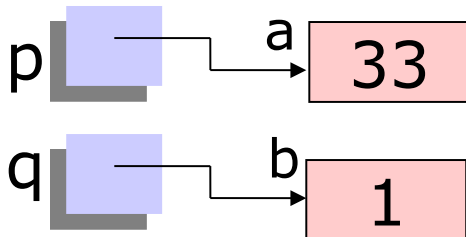
6. `a=33;`



7. `int b=1;`



8. `q=&b;`



NOTA

a, b, p, q : variables
locales

Ejemplo 1:

```
int main() {  
    int x, y, *p, *q;  
    p = &x;  
    *p = 5;  
    q = &y;  
    *q = 23;  
    cout << *p << " " << *q << endl;  
    q = p;  
    *p = 35;  
    cout << *p << " " << *q << endl;  
    q = NULL;  
    cout << x << " " << y << endl;  
    return 0;  
}
```

Ejemplo 2:

```
int main() {  
    int *p, *q, x, y;  
    p = &x;  
    *p = 14;  
    q = p;  
    cout << *p << " " << *q << endl;  
    q = NULL;  
    cout << x << " " << y << endl;  
    return 0;  
}
```

Ejemplo 3:

```
int main() {  
    int x, y, *p, *q;  
    p = &x;  
    *p = 12;  
    q = &y;  
    *q = 23;  
    cout << *p << " " << *q << endl;  
    *p = *q;  
    cout << *p << " " << *q << endl;  
    q = NULL;  
    cout << *p << endl;  
    cout << x << " " << y << endl;  
    return 0;  
}
```

Ejemplo 4:

```
int main() {  
    int x, y, *p, *q;  
    p = &x;  
    *p = 14;  
    q = p;  
    if ( p == q)cout<<"Mensaje 1";  
    q = &y;  
    if ( p == q)cout<<"Mensaje 2";  
    *q = 14;  
    if (p == q) cout<<"Mensaje 3";  
    if (*p == *q)  cout<<"Mensaje 4";  
    return 0;  
}
```

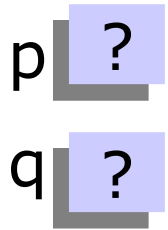
Memoria Dinámica

```
int *p;  
p = new int;  
:::  
delete p;
```

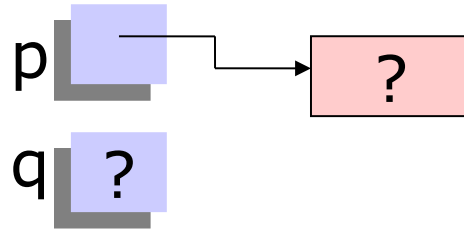
```
int q = new int(3);  
float f = new float(2);  
:::  
delete q;
```

Ejemplo

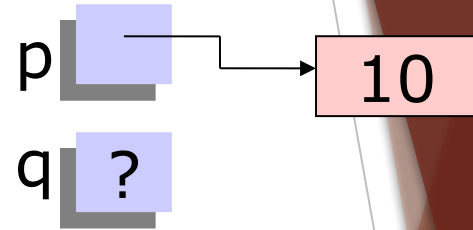
1. `int *p, *q;`



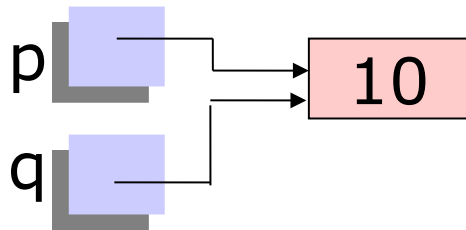
2. `p = new int;`



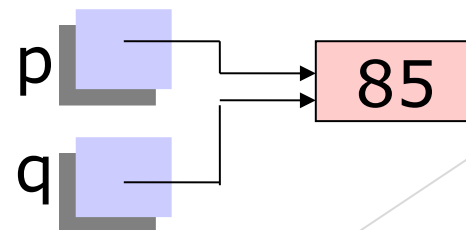
3. `*p = 10;`



4. `q = p;`

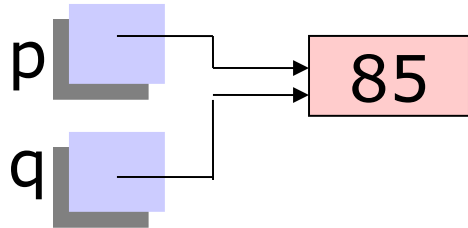


5. `*q = 85;`

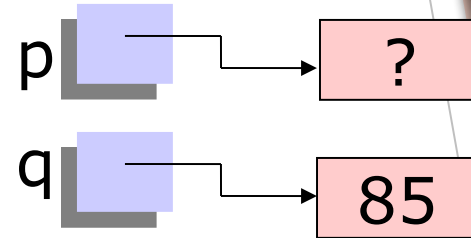


Ejemplo

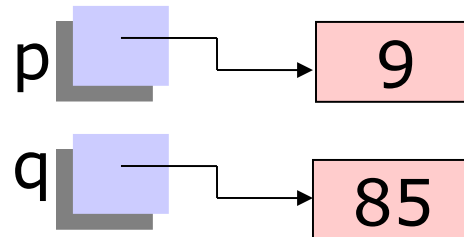
5. *q=85;



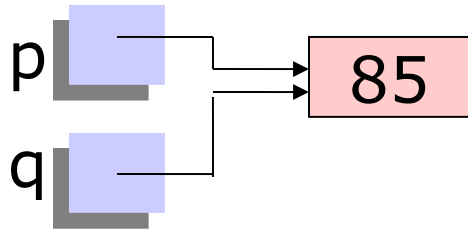
6. p new (int);



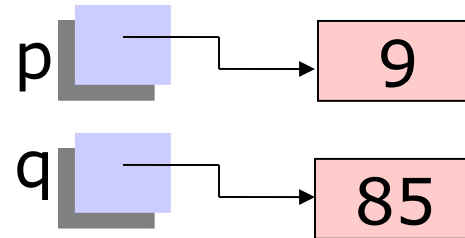
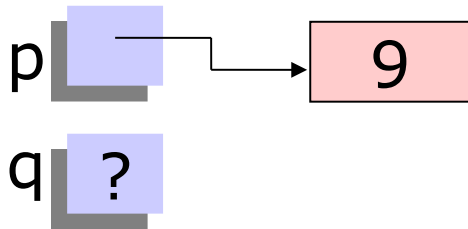
7. *p= 9;



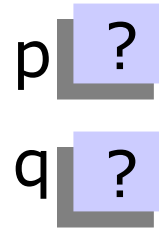
Ejemplo



8. delete *q*;



9. delete *p*;



Memoria Dinámica

```
int *v;  
  
v = new int [100];  
// crea un arreglo dinámico de 100 enteros  
  
delete [] v;  
// libera la memoria ocupada por v
```